AdsKG-Agent: What We Built and How We Built It

Project report · AI Engineering Cohort capstone · Anshika Chaurasia (solo) · 25–30 October 2026

The goal. Build an AI assistant that answers "what happened" questions about ad performance, like *"What was campaign 143747's CTR yesterday, excluding suspicious clicks?"*, by finding the right table among many look-alikes, joining it to the lookup tables it needs, running SQL and showing its work. Then **prove** which design works best with a fair, frozen test.

The approach. Start with the simplest possible version and add one idea at a time, measuring each step: (A) show the AI every table description → (B) basic search → (C) advanced search → (D) search + a map of how tables connect → (E) an agent that uses tools, with memory. Everything was built in 10 milestones (M1–M10), each with a walkthrough notebook.

92.5%	90%	7 / 7	$0.002
best accuracy on 80 unseen test questions (agent + memory)	follow-up questions answered (0% without chat memory)	targets from the design doc met	cost per question (8.6 s median)

The main findings in plain words

- With only 28 tables, simply **showing the AI every table description (A) works very well** (90.4%). Search is only needed once the catalog is too big to show.
- **"Advanced" search tricks made things worse** (C, 68.3%). Rewriting the question and re-ranking didn't help find the tables a join needs.
- The **agent with memory of solved practice examples** is the best (92.5%). It's the only version that handles follow-ups, and its written answers match the data.
- Every claim was checked: 3 runs per version, confidence intervals, significance tests, an AI grader and a human check (kappa 0.65), and a sorted list of all 91 wrong answers.

The big picture

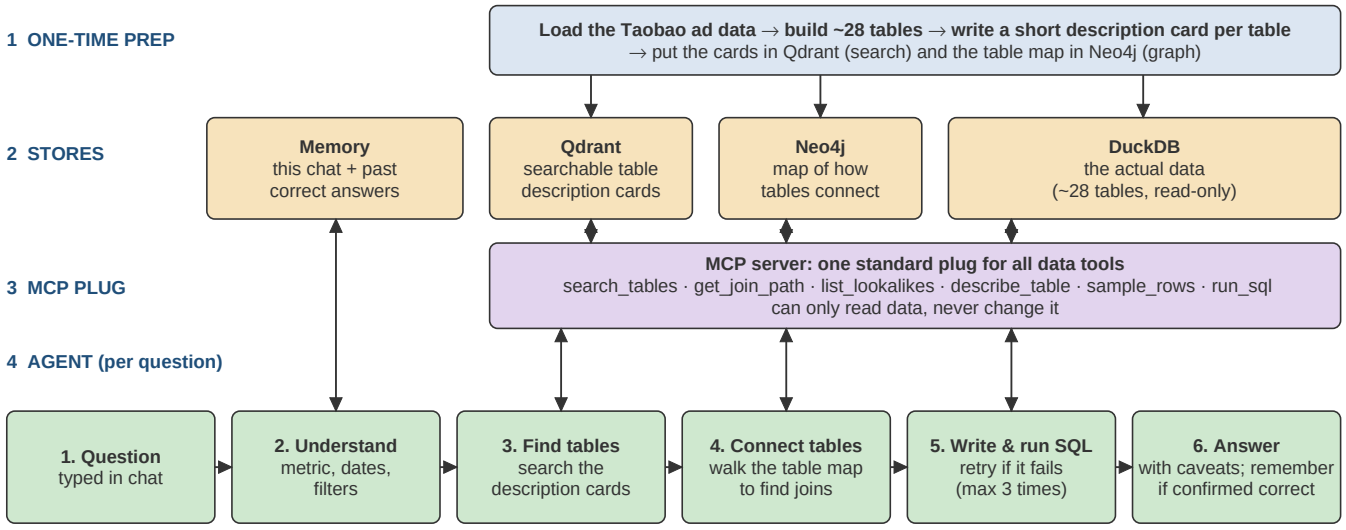


Figure 1. One-time prep builds the tables and their descriptions. For each question, the agent finds and connects tables through one safe MCP server, runs SQL and answers.

Step by step: the 10 milestones

Each milestone had a clear "done when" test, a walkthrough notebook in `notebooks/`, and was finished before the next one started.

M1 · Build the warehouse (28 tables)

What we did	Loaded the public Taobao ad display/click data (26 million ad impressions, ad features, user profiles) and turned it into 28 tables that look alike on purpose: hourly / daily / weekly, all traffic vs valid traffic, impressions vs unique people, live (last 6 hours) vs final.
How	Wrote 19 fixed rules first (`warehouse/rules.md`: time zone, "now" = 13 May 23:59, a suspicious click = a user with more than 5 clicks in one hour). Then one SQL file builds every table in DuckDB (`build_tables.sql`, run by `build.py`).
Result	28 tables in about 15 seconds; a 1.3 GB local database.
Problem → fix	The planned 4th data file (behaviour log) wasn't downloadable → built everything from the 3 available files.

M2 · Describe every table and make them searchable

What we did	Wrote a short "card" per table: what it holds, its grain, freshness, traffic filter, columns, and which tables look like it and how they differ.
How	Hand-written notes → YAML cards → each card turned into a vector (FastEmbed, free local model) → stored in Qdrant, a vector database running as local files.
Result	Search finds the right table in the top 5 for 10 of 10 checks, but ranks a look-alike first in 5 of 10. That's expected: the tables are near-duplicates by design. Deliberately not tuned.

M3 · Create and freeze the test (benchmark)

What we did	120 questions (40 easy, 40 medium with joins, 40 tricky look-alike traps) and 20 three-turn conversations; 40 / 5 for practice, 80 / 15 kept for the final test.
How	Generated from 30 question templates with real IDs (fixed random seed), each with a hand-checked correct SQL. Every answer was recomputed from the raw data a second way (28 of 28 agree). Then the benchmark was **frozen** (a fingerprint makes any later change visible).
Result	A fair exam that every version takes; nothing was changed after building started.
Problem → fix	Some generated questions had empty answers or awkward wording → replaced and fixed before freezing. All of this is disclosed in the README.

M4 · Versions A and B, plus the test harness

What we did	A: give the AI all 28 cards. B: give it only the 5 cards found by search. One script runs any version on any questions and scores it.
How	`harness/run.py configs/A.yaml --split practice`. Scoring = **execution accuracy**: run the AI's SQL and compare the rows with the correct answer. A safety layer (sqlglot) only allows one read-only SELECT.
Result	Practice: A 87.5%, B 70%. B mostly failed on joins: search found the main table but not the lookup table.

M5 · The table map (Neo4j) and version D

What we did	Built a graph of tables, columns, metrics, business words ("views" = impressions) and links: "joins with", "look-alike of", "rolls up to".
How	Neo4j graph database; one Cypher query finds the shortest join route. D = B's search hits + the tables on the join routes + their look-alikes (at most 8 cards).
Result	Join routes 10/10 correct; practice D 85%.
Problem → fix	First D started from 3 search hits and lost tables B had found (77.5%) → start from B's 5 hits.

M6 · One safe plug for all tools: the MCP server

What we did	Put all data tools behind one MCP server: search_tables, get_join_path, list_lookalikes, describe_table, sample_rows, run_sql.
How	FastMCP. Safety lives inside the server so nothing can bypass it: read-only database, one SELECT, 1,000 rows, 20-second timeout, known tables only; every call logged.
Result	6 of 6 tools work; 6 of 6 attack attempts (DELETE, DROP, two statements...) blocked. Any MCP app (e.g. Claude Code) can use it.

M7 · Version E: the agent, with memory

What we did	An agent that thinks, calls a tool, looks at the result and repeats until it can answer. Chat memory for follow-ups, plus example memory: practice questions it got right, recalled as examples.
How	LangGraph loop with hard limits (8 steps, 3 failed SQL runs, a token budget). It discovers the 6 tools from the MCP server. Memory can only learn from practice questions; the harness refuses test questions.
Result	Practice E 85%; follow-up turns 60% with chat memory vs 0% without.
Problem → fix	At the step limit it wrote SQL it had never run → graded only on SQL that really ran. Then it wasted steps hunting for names → added data facts and habits to its instructions (75% → 85%).

M8 · Version C (advanced search) and the bigger-catalog test

What we did	C = B + three tricks: rewrite the question, hybrid search (meaning + keywords), re-rank the shortlist; each switched off one at a time (ablation). Also tested catalogs of 100 and 250 tables.
How	222 extra "distractor" tables from public benchmarks (TPC-DS, TPC-H, BIRD), cards drafted by AI in the same style, added to search and the map only. `--catalog 250` switch.
Result	C was worse than B. A stayed accurate at 250 tables but cost 6× more; search versions cost the same at any size.

M9 · Final exam: 80 test questions, 3 runs each

What we did	Ran all 6 versions 3 times on the unseen test questions, plus 15 test conversations, ablations and bigger catalogs: 35 runs.
How	5 runs in parallel, each with its own copy of the search index (local Qdrant allows one program at a time): about 2 hours instead of 3.5. Then: 95% confidence intervals (bootstrap), McNemar significance tests, an AI grader (gpt-5, blind to version and answer), 30 answers graded by hand, and every wrong answer sorted by type.
Result	E + memory 92.5%, A 90.4%, E 80.0%, D 76.7%, B 75.8%, C 68.3%. Follow-ups 90% vs 28%. All 7 targets met.
Problem → fix	First human-grading pass accepted all 30 (kappa 0.00) → re-checked 2 answers against the reference and changed them to "no" (kappa 0.65). Disclosed.

M10 · Package it

What we did	A chat app, the documentation, this report, the slides and a video script.
How	Streamlit app (`app/streamlit_app.py`) showing each answer's SQL, result and tool calls; PDFs generated from scripts so numbers come straight from the result files.
Result	Everything needed to submit, re-run or present the project.

Results

Version	What it is	Accuracy [95% CI]	Medium	Tricky	Time	Cost
A	all 28 cards, no search	90.4% [84–96]	88%	83%	3.2 s	$0.0012
B	basic search, top 5	75.8% [67–85]	48%	80%	3.3 s	$0.0006
C	rewrite + hybrid + re-rank	68.3% [59–77]	38%	76%	9.4 s	$0.0011
D	search + table map	76.7% [68–85]	52%	81%	3.3 s	$0.0006
E	agent with MCP tools	80.0% [73–87]	54%	92%	8.2 s	$0.0019
E + memory	agent + example memory	92.5% [88–96]	83%	99%	8.6 s	$0.0019

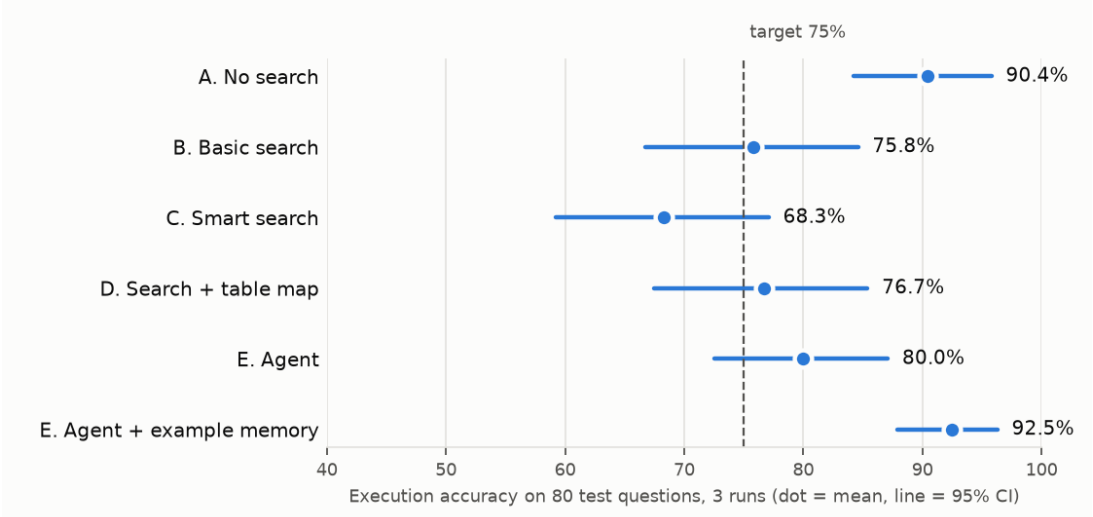


Figure 2. Dot = average of 3 runs on 80 test questions; line = 95% confidence interval (the range where the true accuracy probably lies).

- **Real differences** (McNemar test, $p < 0.05$): A beats B and D; memory makes the agent better (15 questions fixed, 0 broken). B, C, D and the agent without memory are **not** clearly different.
- **Memory honesty check:** 56 of 80 test questions share a template with a practice example in memory. On the 24 that don't, memory still lifts the agent from 72% to 85%, so it's not just copying.
- **Why things fail** (91 wrong answers sorted): wrong calculation (26), search missed the lookup table (18), right value but strict format rule (17), wrong look-alike table (13), "not sure" (13).
- **AI grader vs me:** same verdict on 29 of 30 answers, Cohen's kappa 0.65 (target 0.6).

What we learned

Lesson	Evidence
Build the simple baseline first	A (no search) beat every search version at 28 tables; without it, C or D would have looked like progress
"Advanced" isn't automatically better	rewrite and re-rank each lowered accuracy, on practice and test
The hard part is joins and look-alikes, not SQL	most failures are a missing lookup table or a similar-looking wrong table

Lesson	Evidence
Memory matters most for an agent	example memory +12.5 points; chat memory turns follow-ups from 0% to 90%
Never grade what didn't run	the agent once "answered" with SQL it never executed; grading only executed SQL made it honest
Freeze the test, report everything	every change made on practice questions, and every pivot, is in the README, including negative results

Words used in this project (for beginners)

Word	Meaning
RAG	Retrieval-Augmented Generation: first *find* relevant information (here: table cards), then give it to the AI to write the answer
Embedding / vector	a list of numbers that captures a text's meaning; similar texts get similar numbers, so we can search by meaning
GraphRAG	retrieval that also walks a graph (our table map) to bring in connected items
Agent / ReAct	an AI that loops: think → use a tool → look at the result → repeat
MCP	Model Context Protocol: a standard plug so any AI app can use the same tools
Execution accuracy	run the AI's SQL and check that its rows match the correct answer's rows
Confidence interval	the range where the true score probably lies, given only 80 questions
McNemar test	checks whether two versions really differ, using only questions where they disagree
Cohen's kappa	agreement between two graders beyond luck (1 = perfect, 0 = chance)
Ablation	switch one feature off to see whether it was helping

How to re-run everything

```
python warehouse/build.py · catalog/make_cards.py · catalog/load_qdrant.py · catalog/load_neo4j.py · benchmark/check_gold.py
python harness/run.py configs/<A|B|C|D|E|E_memory>.yaml --split test [--conversations] [--catalog 250]
python harness/stats.py · harness/chart.py · harness/judge.py · harness/failures.py
streamlit run app/streamlit_app.py
```

Full commands, setup and every number are in the README; each milestone has a walkthrough notebook.